



РЭУ.РФ
РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ Г.В. ПЛЕХАНОВА

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Ощепков Михаил Алексеевич
(подпись) (фамилия, имя, отчество)

Группы ИПо-32

Руководитель практической подготовки от профильной организации

Преподаватель
(должность)

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

МП «26» июня 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЙ К ПРОГРАММНЫМ МОДУЛЯМ ..	5
1.1. Сбор информации о существующем состоянии продукта.....	5
1.2. Разработка технического задания.....	6
РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ .	8
2.1. Процесс разработки программного обеспечения	8
2.2. Среда программирования	9
2.3. Диаграмма классов.....	9
2.4. Разработка программного модуля	12
РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ	15
3.1. Процесс отладки.....	15
РАЗДЕЛ 4. РАЗРАБОТКА ТЕСТОВЫХ НАБОРОВ И СЦЕНАРИЕВ	17
4.1. Процесс проведения тестирования ПО	17
4.2. Создание unit-тестов	18
РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ НА СООТВЕТСТВИЕ СТАНДАРТАМ КОДИРОВАНИЯ.....	20
5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования	20
РАЗДЕЛ 6. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ ПРИЛОЖЕНИЕМ	22
6.1. Главное окно приложения.....	22
6.2. Добавление новой задачи	23
6.3. Редактирование задачи	23
6.4. Фильтрация задач по статусу.....	24
6.5. Поиск задач по названию и описанию	25
6.6. Сортировка задач	25
6.7. Сохранение и загрузка данных из файла	26
6.8. Индикация просроченных задач.....	26
ЗАКЛЮЧЕНИЕ	28
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	30
ТРЕБОВАНИЕ К НАПИСАНИЮ ПРОГРАММНОГО МОДУЛЯ.....	32

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение обучающимися всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приёмов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчёт о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: разработка программного обеспечения; выполнение отладки программного модуля с использованием специализированных программных средств; осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования.

В качестве объекта разработки было выбрано приложение для управления задачами OshchepkovTaskManager — настольная программа на платформе .NET 8 с пользовательским интерфейсом, реализованным по технологии WPF (Windows Presentation Foundation). Выбор данной темы обусловлен возможностью продемонстрировать на практике все этапы разработки программного обеспечения: от анализа требований до тестирования готового продукта.

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент;
2. Выполнять интеграцию модулей в программное обеспечение;
3. Выполнять отладку программного модуля с использованием специализированных программных средств;
4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
5. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;
6. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
7. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности;
8. Планировать и реализовывать собственное профессиональное и личностное развитие;
9. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами;
10. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учётом особенностей социального и культурного контекста;
11. Использовать информационные технологии в профессиональной деятельности;
12. Пользоваться профессиональной документацией на государственном и иностранном языках.

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЙ К ПРОГРАММНЫМ МОДУЛЯМ

1.1. Сбор информации о существующем состоянии продукта

Перед началом разработки была проведена постановка задач. Заказчиком (руководителем практики) было сформулировано задание: разработать приложение для управления задачами (планировщик задач) с возможностью добавления, просмотра, редактирования, удаления задач, их фильтрации по статусу, поиска по тексту, а также сохранения и загрузки данных из файла.

Описание бизнес-процесса в состоянии «AS IS» (до автоматизации): на момент постановки задачи у пользователя отсутствовал инструмент для структурированного учёта задач. Списки дел вели вручную (в текстовых файлах, на бумаге или в сторонних универсальных приложениях), что не позволяло автоматически отслеживать приоритеты, сроки выполнения и статус задач, а также не давало возможности быстро находить нужную задачу среди большого количества записей.

Описание процесса модернизации, состояние «TO BE» (после автоматизации): пользователь получает специализированное настольное приложение, в котором каждая задача описывается набором атрибутов (название, описание, приоритет, срок выполнения, статус, признак важности). Приложение позволяет мгновенно отфильтровать список по статусу, найти задачу по ключевому слову, отсортировать список по приоритету или сроку, а также видеть статистику по всем задачам (сколько новых, в процессе, завершено, просрочено) без необходимости вручную пересчитывать данные.

Сравнительный анализ процессов «AS IS» и «TO BE» показал, что усовершенствование заключается в следующем: исключена ручная сортировка и поиск задач, добавлена визуальная индикация просроченных задач, автоматизирован подсчёт статистики, реализовано долговременное хранение

данных в структурированном формате (JSON) с возможностью переноса между компьютерами.

Задачи, которые необходимо автоматизировать: ведение перечня задач с атрибутами; изменение статуса и редактирование задачи; фильтрация и поиск; сортировка; сохранение и восстановление данных между запусками приложения; отображение статистики.

1.2. Разработка технического задания

Основные требования к техническому заданию были сформулированы в соответствии с ГОСТ 19.201-78 «Единая система программной документации. Техническое задание. Требования к содержанию и оформлению» и с учётом рекомендаций ГОСТ 34.602-2020 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

Стандарты ГОСТ 19 (Единая система программной документации, ЕСПД) регламентируют состав и содержание документации на стадии разработки программного обеспечения, включая структуру технического задания, программы и методики испытаний, текстов программ и эксплуатационной документации. Стандарты серии ГОСТ 34 регламентируют требования к автоматизированным системам в целом, включая этапы создания, виды обеспечения и состав технической документации.

Содержание технического задания на разработку приложения OshcherkovTaskManager включает следующие разделы: наименование и область применения; основание для разработки; назначение разработки; технические требования к программе; стадии и этапы разработки; порядок контроля и приёмки.

Структура технического задания, использованная при разработке, включала следующие функциональные требования к приложению:

- добавление новой задачи с указанием названия, описания, приоритета, срока выполнения и статуса;
- просмотр списка всех задач в табличном представлении;
- фильтрация задач по статусу: «Новая», «В процессе», «Завершена», «Отменена»;
- поиск задач по названию или описанию в режиме реального времени;
- редактирование существующих задач;
- удаление задач с подтверждением действия;
- сохранение и загрузка задач в/из файла формата JSON;
- сортировка задач по приоритету и сроку выполнения (по возрастанию и убыванию);
- визуальная пометка важных задач (звёздочка и цветовая индикация);
- отображение статистики по задачам (всего, новых, в процессе, завершено, просрочено).

Порядок документирования требований предусматривал фиксацию каждого требования в виде отдельного пункта технического задания с указанием способа его проверки (приёмочного критерия). Например, для требования «фильтрация по статусу» критерием приёмки являлось отображение в списке только тех задач, статус которых совпадает с выбранным значением в выпадающем списке фильтра.

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1. Процесс разработки программного обеспечения

Для разработки приложения была выбрана каскадная (водопадная) модель процесса разработки. Данная модель предполагает последовательное прохождение этапов: анализ требований, проектирование, реализация, тестирование, внедрение, сопровождение, причём переход к следующему этапу осуществляется только после завершения предыдущего.

Рассматривались также альтернативные модели: V-образная модель разработки (расширение каскадной модели, предусматривающее параллельное планирование тестов на каждом этапе разработки), модель прототипирования (создание упрощённой рабочей версии продукта для уточнения требований), модель быстрой разработки RAD (с акцентом на быстрое итеративное создание прототипов), итерационная модель (разбиение разработки на несколько повторяющихся циклов) и спиральная модель (объединяющая итеративность с анализом рисков на каждом витке).

Обоснование выбора каскадной модели: задание на практику содержало чётко зафиксированный и заранее известный перечень требований (добавление, просмотр, фильтрация, поиск, редактирование, удаление, сохранение/загрузка, тесты), которые не менялись в процессе разработки. Сжатые сроки прохождения практики делали более выгодным линейный процесс без множества итераций согласования прототипов с заказчиком. Кроме того, каскадная модель хорошо документируется и облегчает контроль выполнения этапов работ в рамках учебной практики.

Инструменты, использованные для создания продукта: интегрированная среда разработки Microsoft Visual Studio 2022; платформа .NET 8 SDK; система контроля версий Git и хостинг GitHub для хранения исходного кода; фреймворк

модульного тестирования xUnit; библиотека System.Text.Json для сериализации данных.

2.2. Среда программирования

В качестве среды программирования использовалась Microsoft Visual Studio 2022 — интегрированная среда разработки, поддерживающая создание приложений на языке C# для платформы .NET. Среда предоставляет инструменты отладки, статического анализа кода, рефакторинга, а также встроенный конструктор диаграмм классов и интеграцию со средствами модульного тестирования.

Язык программирования — C# (версия совместима с .NET 8). Выбор данного языка обусловлен его объектно-ориентированной природой, строгой типизацией, наличием LINQ (Language Integrated Query) для удобной работы с коллекциями данных и широкой поддержкой создания как консольных, так и оконных приложений с графическим интерфейсом.

Для построения пользовательского интерфейса использовалась технология WPF (Windows Presentation Foundation) — фреймворк для создания настольных приложений с графическим интерфейсом на платформе .NET, использующий декларативный язык разметки XAML для описания внешнего вида окон и элементов управления.

2.3. Диаграмма классов

Решение OshchepkovTaskManager состоит из трёх проектов: библиотеки бизнес-логики OshchepkovTaskManager.Core, WPF-приложения OshchepkovTaskManager.App и проекта модульных тестов OshchepkovTaskManager.Tests. Такое разделение соответствует требованию задания о выделении основной логики приложения в отдельную динамическую библиотеку (DLL), отделённую от пользовательского интерфейса.

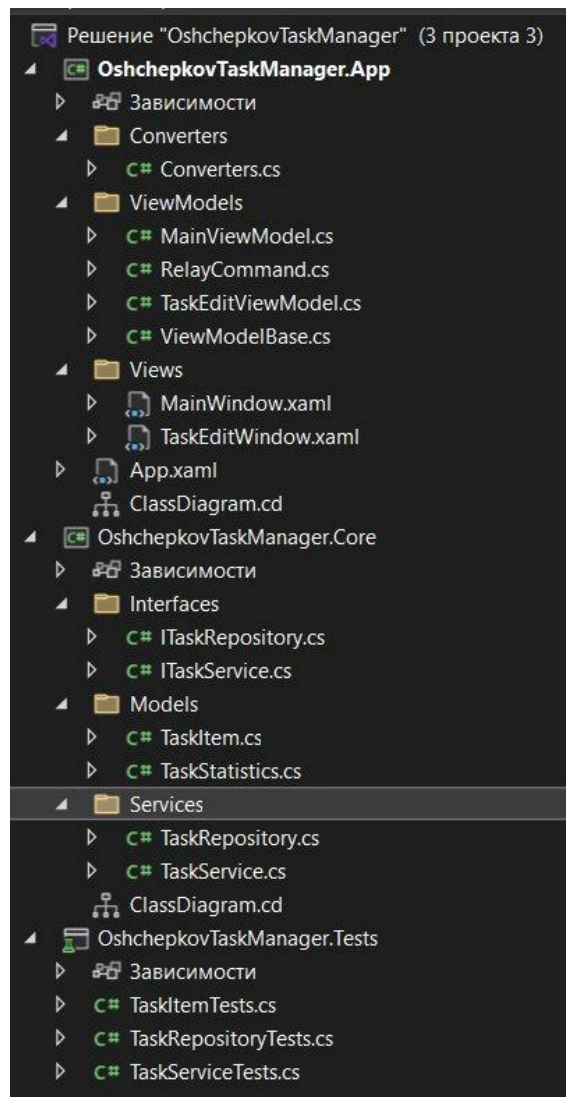


Рисунок 1 — Структура решения в Solution Explorer

Диаграмма классов библиотеки `OshchepkovTaskManager.Core` была построена средствами встроенного конструктора Visual Studio 2022 (Class Designer). Диаграмма отражает состав классов, перечислений и интерфейсов, а также их свойства и методы.

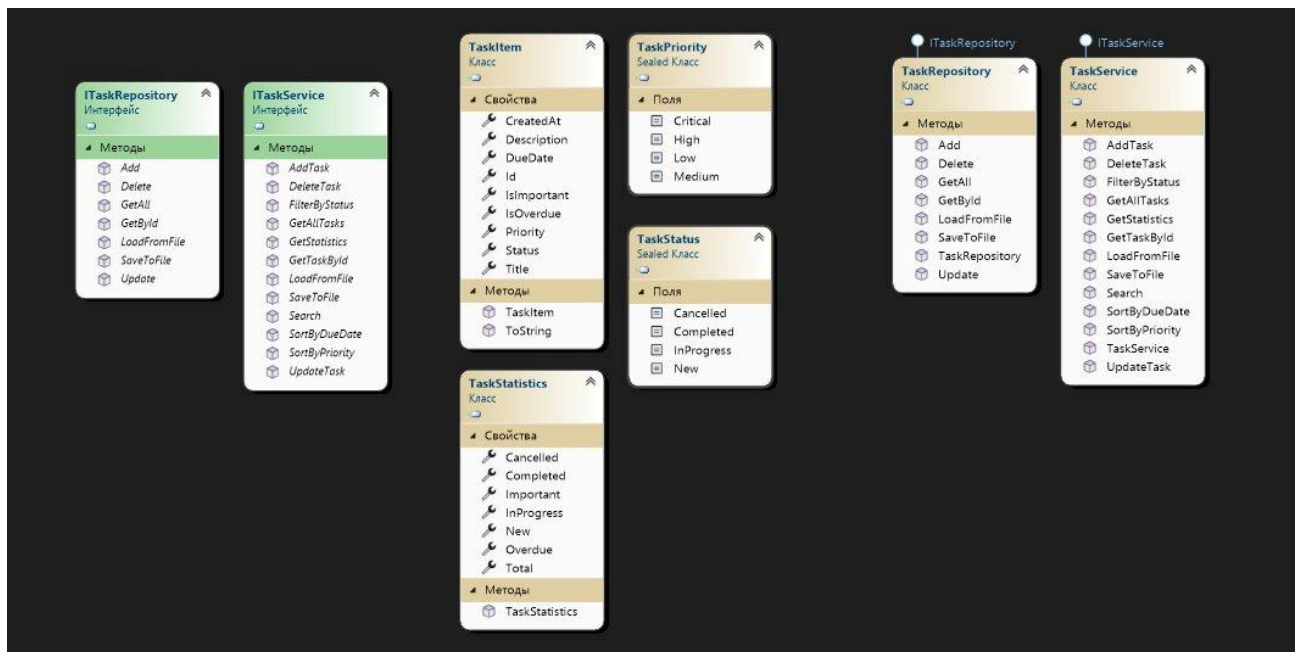


Рисунок 2 — Диаграмма классов библиотеки OshchepkovTaskManager.Core

Состав классов и интерфейсов библиотеки:

- TaskItem — модель данных задачи. Содержит свойства Id (уникальный идентификатор), Title (название), Description (описание), Priority (приоритет), DueDate (срок выполнения), Status (статус), IsImportant (признак важности), CreatedAt (дата создания), а также вычисляемое свойство IsOverdue, определяющее, является ли задача просроченной;
- TaskStatistics — модель данных статистики с полями-счётчиками: Total, New, InProgress, Completed, Cancelled, Overdue, Important;
- TaskPriority — перечисление приоритетов задачи: Low (низкий), Medium (средний), High (высокий), Critical (критический);
- TaskStatus — перечисление статусов задачи: New (новая), InProgress (в процессе), Completed (завершена), Cancelled (отменена);
- ITaskRepository — интерфейс репозитория, объявляющий методы доступа к коллекции задач (Add, Update, Delete, GetAll, GetById) и методы файлового ввода-вывода (SaveToFile, LoadFromFile);
- ITaskService — интерфейс сервиса бизнес-логики, объявляющий методы CRUD-операций, методы поиска (Search), фильтрации (FilterByStatus),

сортировки (SortByPriority, SortByDueDate) и получения статистики (GetStatistics);

- TaskRepository — класс, реализующий интерфейс ITaskRepository. Хранит коллекцию задач в оперативной памяти (List<TaskItem>) и обеспечивает сериализацию/десериализацию данных в формат JSON с использованием библиотеки System.Text.Json;
- TaskService — класс, реализующий интерфейс ITaskService. Делегирует операции хранения данных объекту ITaskRepository и реализует бизнес-логику фильтрации, поиска и сортировки с использованием LINQ-запросов.

Взаимодействие компонент: класс TaskService принимает в конструкторе экземпляр ITaskRepository (паттерн внедрения зависимостей, Dependency Injection), что обеспечивает слабую связанность компонентов — реализацию репозитория можно заменить (например, на версию с хранением в базе данных) без изменения кода сервиса.

2.4. Разработка программного модуля

Разработка программного модуля выполнялась согласно техническому заданию (приложение 1 к настоящему отчёту). Реализация была разделена на следующие этапы: создание модели данных, создание интерфейсов, реализация репозитория и сервиса в библиотеке Core, разработка пользовательского интерфейса в проекте App, написание модульных тестов в проекте Tests.

Этапы разработки ПО: на этапе выбора шаблона был создан каркас решения из трёх проектов в Visual Studio (библиотека классов, WPF-приложение, проект тестов xUnit) с соответствующими ссылками между проектами. На этапе составления схемы информационной базы определена структура хранения данных — единая JSON-коллекция объектов TaskItem, сохраняемая в выбираемый пользователем файл. На этапе документирования системы подготовлена настоящая сопроводительная документация с диаграммой классов

и описанием функциональности. Этап тестирования включал написание и выполнение модульных тестов (подробно описан в разделе 4 настоящего отчёта). Этап сопровождения системы предполагает дальнейшую поддержку и расширение функциональности приложения (например, добавление синхронизации с облачным хранилищем или многопользовательского режима).

Формы для ввода, просмотра и редактирования данных реализованы в виде двух окон WPF: главного окна (MainWindow), содержащего список всех задач, панель фильтрации и поиска, а также блок статистики, и диалогового окна (TaskEditWindow), используемого как для добавления новой задачи, так и для редактирования существующей.

Этапы разработки интерфейса: на этапе проектирования была определена общая компоновка окна — боковая панель управления слева и рабочая область со списком задач справа. На этапе прототипирования создана базовая разметка XAML без окончательного оформления. На этапе стилизации были применены единые стили оформления элементов управления (кнопок, текстовых полей, выпадающих списков), выбрана тёмная цветовая палитра с фиолетовым акцентным цветом, обеспечивающая современный и комфортный для глаз внешний вид приложения.

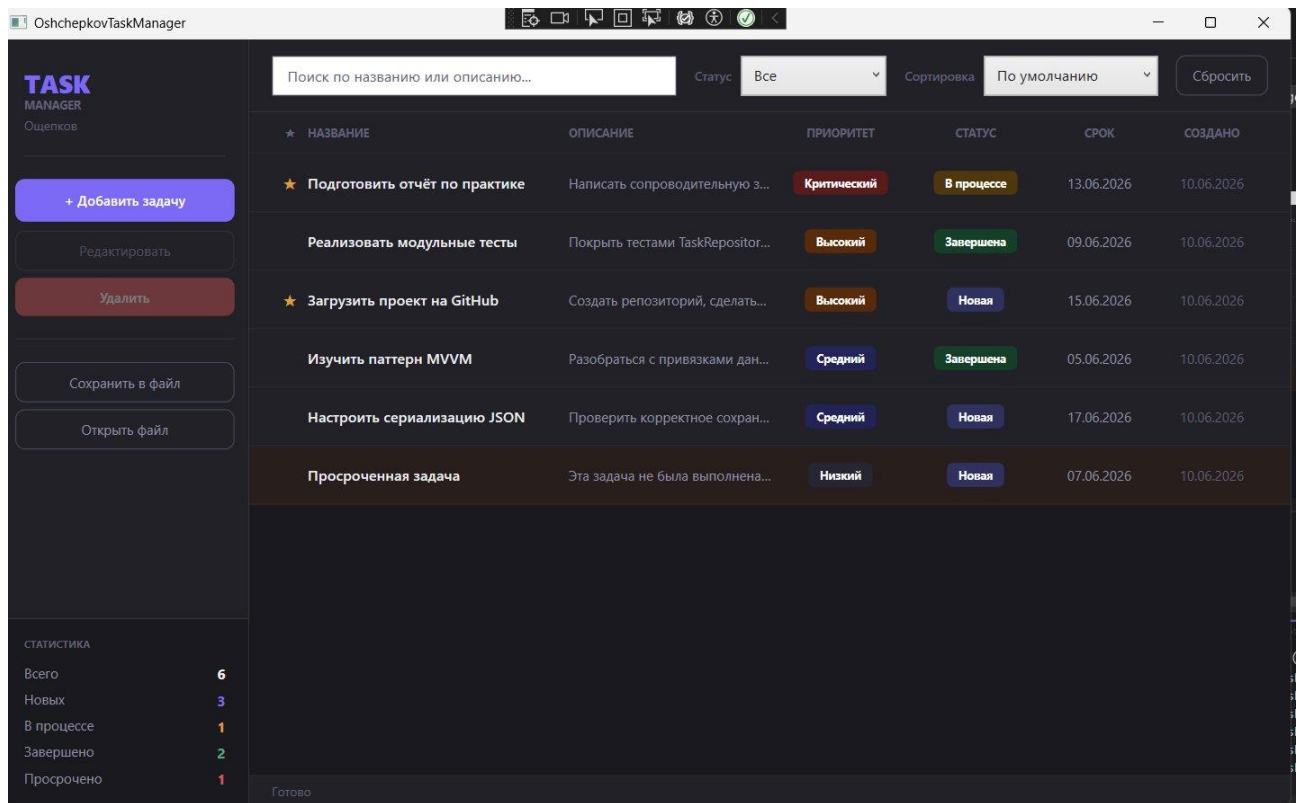


Рисунок 3 — Главное окно приложения OshchepkovTaskManager

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ

3.1. Процесс отладки

Отладка программного модуля производилась с использованием встроенного отладчика Visual Studio Debugger, входящего в состав среды разработки Microsoft Visual Studio 2022. Отладчик позволяет устанавливать точки останова (breakpoints), пошагово выполнять код (step into, step over, step out), просматривать значения переменных в режиме реального времени и анализировать стек вызовов.

В процессе разработки было обнаружено и устранено несколько типов ошибок. Первый тип — ошибки компиляции, связанные с конфликтом имён типов: класс `TaskStatus`, определённый в библиотеке `Core` (`OshchepkovTaskManager.Core.Models.TaskStatus`), конфликтовал с одноимённым системным типом `System.Threading.Tasks.TaskStatus`, неявно подключаемым при использовании пространства имён `System.Windows.Input` в WPF-проекте. Конфликт устранён добавлением псевдонима пространства имён (`using TaskStatus = OshchepkovTaskManager.Core.Models.TaskStatus`) во всех файлах, где использовался данный тип.

Второй тип ошибок — ошибки разметки XAML (код MC3072), связанные с использованием свойств элементов управления, недоступных в применяемой версии WPF (например, свойство `Spacing` у контейнера `StackPanel` и свойство `LetterSpacing` у текстового блока `TextBlock`). Данные ошибки устранялись заменой неподдерживаемых свойств на эквивалентные конструкции — в частности, интервалы между элементами реализовывались через свойство `Margin` отдельных дочерних элементов.

Третий тип — логические ошибки отображения, в частности проблема видимости текста в полях ввода, когда цвет текста совпадал с цветом фона элемента из-за особенностей наследования свойства `Foreground` внутри шаблона элемента `TextBox`. Ошибка была устранена путём отказа от переопределения

визуального шаблона (ControlTemplate) поля ввода и использования стандартного шаблона WPF с явным указанием контрастных цветов фона и текста.

Кроме того, в процессе сборки проекта была выявлена и устранена уязвимость в зависимости NuGet-пакета System.Text.Json версии 8.0.0 (код предупреждения NU1903, ссылки на бюллетени безопасности GHSA-8g4q-xg66-9fp4 и GHSA-hh2w-p6rv-4g7w). Уязвимость устранена обновлением пакета до версии 8.0.5, в которой соответствующие проблемы безопасности устранены разработчиком библиотеки.

Для верификации правильности работы методов библиотеки в процессе отладки также использовались точки останова в методах TaskService (например, в методах Search и GetStatistics) с проверкой содержимого коллекций на промежуточных шагах LINQ-запросов.

РАЗДЕЛ 4. РАЗРАБОТКА ТЕСТОВЫХ НАБОРОВ И СЦЕНАРИЕВ

4.1. Процесс проведения тестирования ПО

Тестирование разработанного программного обеспечения проводилось на двух уровнях: модульное (unit) тестирование классов библиотеки `OshcherkovTaskManager.Core` и функциональное тестирование пользовательского интерфейса приложения путём ручной проверки сценариев использования.

Модульное тестирование охватывает три ключевых класса библиотеки: `TaskItem` (модель данных), `TaskRepository` (репозиторий с хранением в памяти и сериализацией) и `TaskService` (сервис бизнес-логики). Для каждого класса разработан отдельный набор тестов, проверяющий как штатные сценарии работы (positive cases), так и обработку ошибочных ситуаций (negative cases).

Тестовые сценарии для класса `TaskItem` включают проверку уникальности идентификатора у разных экземпляров задачи, корректность значений свойств по умолчанию (статус «Новая», приоритет «Средний», признак важности «выключен»), а также корректность вычисления свойства `IsOverdue` для различных комбинаций статуса и срока выполнения задачи.

Тестовые сценарии для класса `TaskRepository` включают проверку добавления задачи (в том числе обработку исключений при передаче null-значения или пустого названия), получения задачи по идентификатору (существующему и несуществующему), обновления и удаления задачи, а также сохранения и загрузки данных из файла (round-trip тест, проверяющий, что сохранённые и затем загруженные данные идентичны исходным).

Тестовые сценарии для класса `TaskService` включают проверку фильтрации по статусу, регистронезависимого поиска по названию и описанию, сортировки по приоритету и сроку выполнения в обоих направлениях (по возрастанию и по

убыванию), а также корректность подсчёта статистики при различных комбинациях статусов задач.

4.2. Создание unit-тестов

Для создания модульных тестов использован фреймворк xUnit.net — один из наиболее распространённых фреймворков тестирования для платформы .NET. Тестовый проект OshchepkovTaskManager.Tests подключён к проекту библиотеки OshchepkovTaskManager.Core через ссылку на проект (ProjectReference) и использует пакеты Microsoft.NET.Test.Sdk, xunit и xunit.runner.visualstudio.

Каждый тестовый метод оформлен атрибутом [Fact] и содержит три логические части: подготовку тестовых данных (Arrange), вызов проверяемого метода (Act) и проверку результата с помощью методов класса Assert (Assert). Для проверки исключительных ситуаций используется конструкция Assert.Throws, проверяющая, что метод выбрасывает исключение ожидаемого типа.

Всего было разработано 39 модульных тестов, распределённых по трём тестовым классам: TaskItemTests (9 тестов), TaskRepositoryTests (13 тестов) и TaskServiceTests (17 тестов). Запуск тестов осуществлялся средствами Test Explorer в Visual Studio 2022. Результаты запуска всех тестов представлены на рисунке 4.

▲	✓	OshchepkovTaskManager.Tests (39)	98 мс
▲	✓	OshchepkovTaskManager.Tests (39)	98 мс
▲	✓	TaskItemTests (9)	19 мс
	✓	IsOverdue_FutureDueDate_Not...	< 1 мс
	✓	IsOverdue_PastDueDate_Cancel...	< 1 мс
	✓	IsOverdue_PastDueDate_Compl...	< 1 мс
	✓	IsOverdue_PastDueDate_New_I...	< 1 мс
	✓	NewTask_DefaultPriority_IsMedi...	2 мс
	✓	NewTask_DefaultStatus_IsNew	5 мс
	✓	NewTask_HasUniqueld	2 мс
	✓	NewTask_IsNotImportant_ByDe...	< 1 мс
	✓	ToString_ContainsTitleAndStatus	10 мс
▲	✓	TaskRepositoryTests (13)	59 мс
	✓	Add_EmptyTitle_ThrowsArgume...	3 мс
	✓	Add_MultipleItems_AllPresent	< 1 мс
	✓	Add_NullTask_ThrowsArgument...	< 1 мс
	✓	Add_ValidTask_IncreasesCount	< 1 мс
	✓	Delete_ExistingId_RemovesTask	< 1 мс
	✓	Delete_NonExistingId_ThrowsK...	1 мс
	✓	GetById_ExistingId_ReturnsTask	< 1 мс
	✓	GetById_NonExistingId_Returns...	< 1 мс
	✓	LoadFromFile_NonExistingFile_...	< 1 мс
	✓	SaveAndLoad_RoundTrip_Prese...	55 мс
	✓	SaveToFile_EmptyPath_ThrowsA...	< 1 мс
	✓	Update_ExistingTask_ChangesD...	< 1 мс
	✓	Update_NonExistingTask_Throw...	< 1 мс
▶	✓	TaskServiceTests (17)	20 мс

Рисунок 4 — Результаты запуска модульных тестов в Test Explorer (39 тестов, все пройдены успешно)

Все 39 тестов успешно пройдены (отмечены зелёными значками), что подтверждает корректность реализации основной бизнес-логики библиотеки OshchepkovTaskManager.Core.

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ НА СООТВЕТСТВИЕ СТАНДАРТАМ КОДИРОВАНИЯ

5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

Виды инспекции, применённые к разработанному программному коду: статический анализ кода средствами компилятора и анализаторов Visual Studio (выявление предупреждений компилятора, неиспользуемых директив `using`, нарушений правил `Nullable Reference Types`); ручной код-ревью с проверкой соответствия общепринятым правилам оформления кода C#; проверка соответствия структуры решения принципам разделения ответственности (*Separation of Concerns*) между слоями приложения.

Актуальные стандарты кодирования, применённые при разработке: рекомендации Microsoft по стилю кода C# (*C# Coding Conventions*), предполагающие использование *PascalCase* для имён классов, методов и публичных свойств, *camelCase* с префиксом подчёркивания для приватных полей, а также обязательное применение пространств имён, соответствующих структуре проекта. Также использовались принципы SOLID, в частности принцип единственной ответственности (*Single Responsibility Principle*) — каждый класс библиотеки выполняет одну чётко определённую функцию (*TaskRepository* отвечает только за хранение данных, *TaskService* — только за бизнес-логику), и принцип инверсии зависимостей (*Dependency Inversion Principle*) — класс *TaskService* зависит от абстракции *ITaskRepository*, а не от конкретной реализации.

Программы, при помощи которых осуществлялась инспекция кода: встроенный анализатор кода Visual Studio 2022 (*Roslyn Analyzers*), отображающий предупреждения и ошибки непосредственно в окне *Error List* в процессе редактирования и сборки проекта; компилятор C# (*Roslyn Compiler*), выполняющий проверку типов и синтаксиса на этапе сборки решения.

В результате инспектирования были выявлены и устранены следующие несоответствия: неиспользуемые директивы `using`, отсутствие явных модификаторов доступа у некоторых членов класса, а также описанные в разделе 3 настоящего отчёта ошибки конфликта имён типов и недоступных в WPF свойств XAML-разметки. После устранения всех замечаний сборка проекта выполняется без ошибок и предупреждений компилятора, за исключением информационных сообщений NuGet, относящихся к версиям пакетов.

РАЗДЕЛ 6. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ ПРИЛОЖЕНИЕМ

В данном разделе представлено описание основных пользовательских сценариев работы с разработанным приложением, иллюстрированное скриншотами интерфейса.

6.1. Главное окно приложения

При запуске приложения открывается главное окно, состоящее из двух областей: боковой панели управления (слева), содержащей кнопки операций над задачами и блок статистики, и рабочей области (справа), содержащей панель поиска и фильтрации, а также список задач. При первом запуске список автоматически заполняется демонстрационными задачами для удобства проверки функциональности.

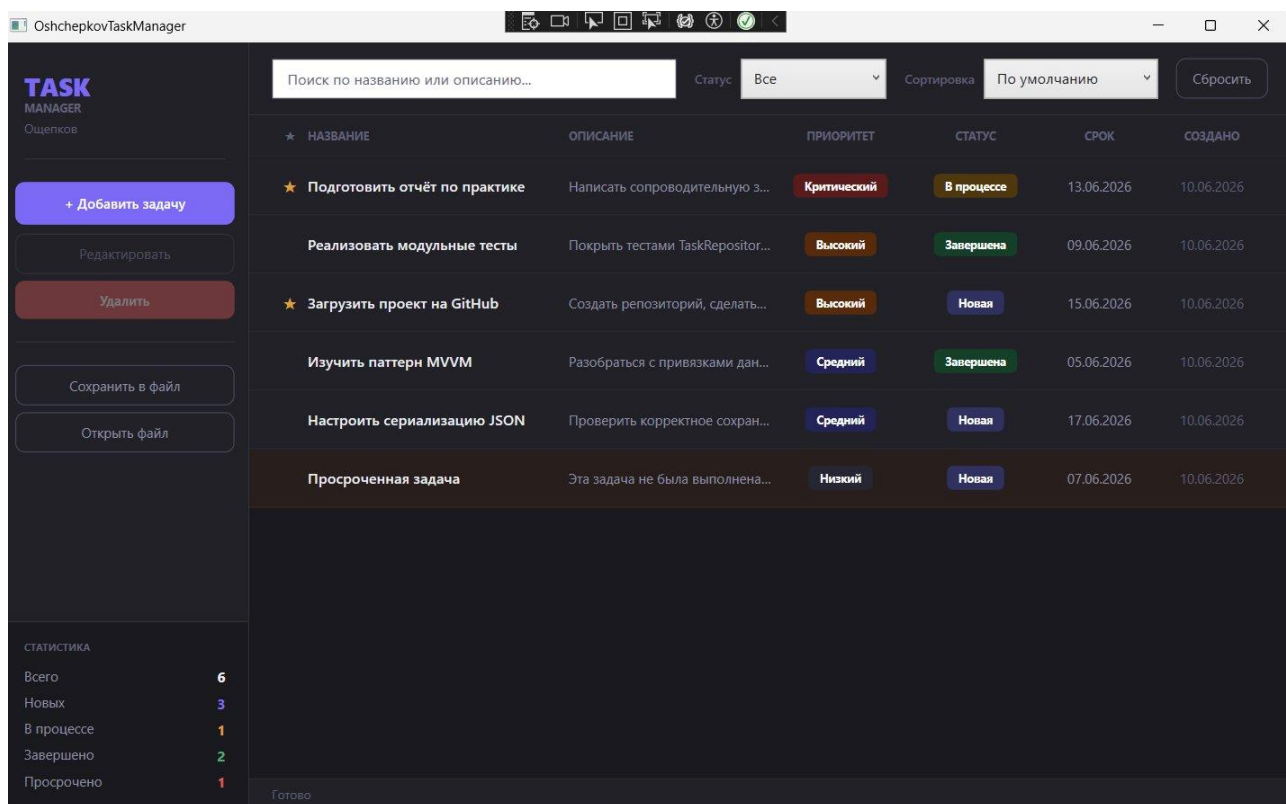


Рисунок 5 — Главное окно приложения со списком задач

6.2. Добавление новой задачи

Для добавления новой задачи необходимо нажать кнопку «Добавить задачу» в боковой панели. Открывается диалоговое окно «Новая задача» с полями для ввода названия (обязательное поле), описания, выбора приоритета и статуса из выпадающих списков, выбора срока выполнения с помощью календаря, а также переключателем «Важная задача». После заполнения формы нажатие кнопки «Сохранить» добавляет задачу в список.

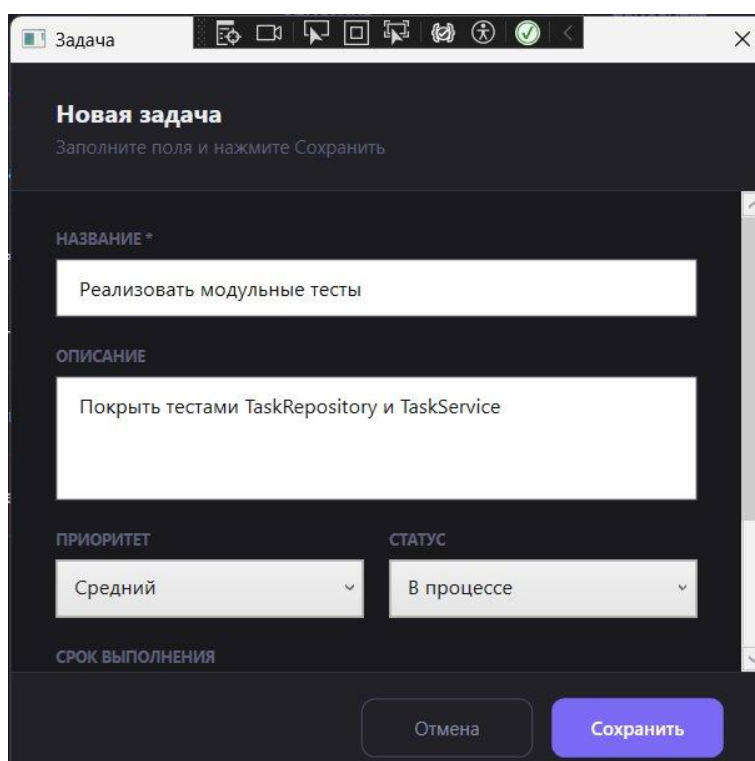


Рисунок 6 — Диалоговое окно добавления новой задачи

6.3. Редактирование задачи

Для редактирования существующей задачи необходимо выбрать её в списке одиночным щелчком мыши и нажать кнопку «Редактировать» (либо выполнить двойной щелчок непосредственно по строке задачи). Открывается то же диалоговое окно, но с заголовком «Редактировать задачу» и предварительно заполненными текущими значениями полей. После внесения изменений необходимо нажать «Сохранить».

Рисунок 7 — Диалоговое окно редактирования задачи

6.4. Фильтрация задач по статусу

Панель фильтрации, расположенная над списком задач, содержит выпадающий список «Статус» с вариантами: «Все», «Новая», «В процессе», «Завершена», «Отменена». При выборе значения список задач немедленно обновляется, показывая только задачи с выбранным статусом. На рисунке 8 представлен пример поиска по ключевому слову «Загрузить», в результате которого отображается единственная соответствующая задача с пометкой важности.

★ НАЗВАНИЕ	ОПИСАНИЕ	ПРИОРИТЕТ	СТАТУС	СРОК	СОЗДАНО
Реализовать модульные тесты	Покрыть тестами TaskRepositor...	Высокий	Завершена	09.06.2026	10.06.2026
Изучить паттерн MVVM	Разобраться с привязками дан...	Средний	Завершена	05.06.2026	10.06.2026

Рисунок 8 — Панель фильтрации и результат поиска по слову «Загрузить»

6.5. Поиск задач по названию и описанию

Поле поиска, расположенное в левой части панели фильтрации, осуществляет регистронезависимый поиск одновременно по названию и описанию задачи в режиме реального времени — результат обновляется после каждого введённого символа без необходимости нажатия дополнительных кнопок.

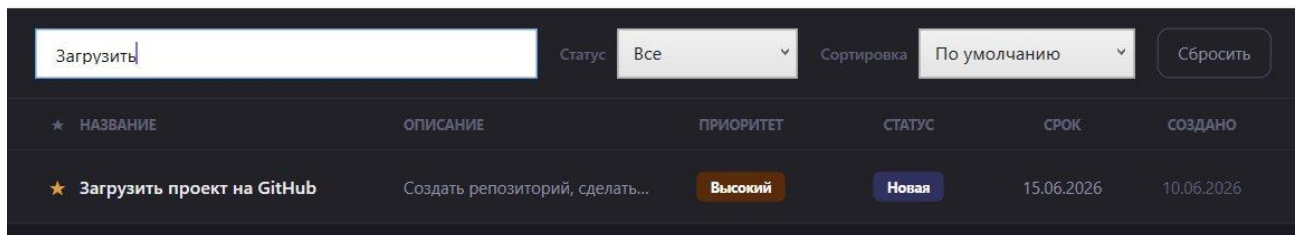


Рисунок 9 — Поле поиска по названию или описанию задачи

6.6. Сортировка задач

Выпадающий список «Сортировка» позволяет упорядочить список задач по следующим критериям: «По умолчанию» (порядок добавления), «По приоритету ↓» (от критического к низкому), «По приоритету ↑» (от низкого к критическому), «По сроку ↑» (от ближайшего срока к отдалённому) и «По сроку ↓» (в обратном порядке). На рисунке 10 представлен результат сортировки по приоритету в порядке возрастания — список начинается с задачи низкого приоритета.

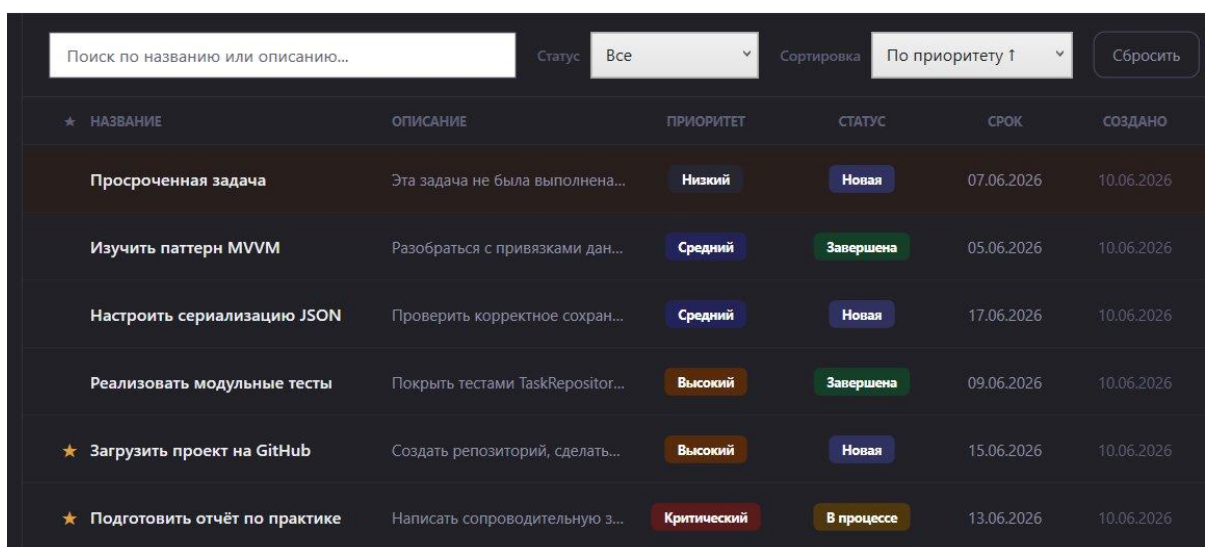


Рисунок 10 — Список задач, отсортированный по приоритету (по возрастанию)

6.7. Сохранение и загрузка данных из файла

Кнопка «Сохранить в файл» в боковой панели открывает стандартный диалог выбора места сохранения, предлагая по умолчанию формат JSON (JavaScript Object Notation). Кнопка «Открыть файл» открывает диалог выбора ранее сохранённого JSON-файла для восстановления списка задач.



Рисунок 11 — Фрагмент диалога сохранения задач в JSON-файл

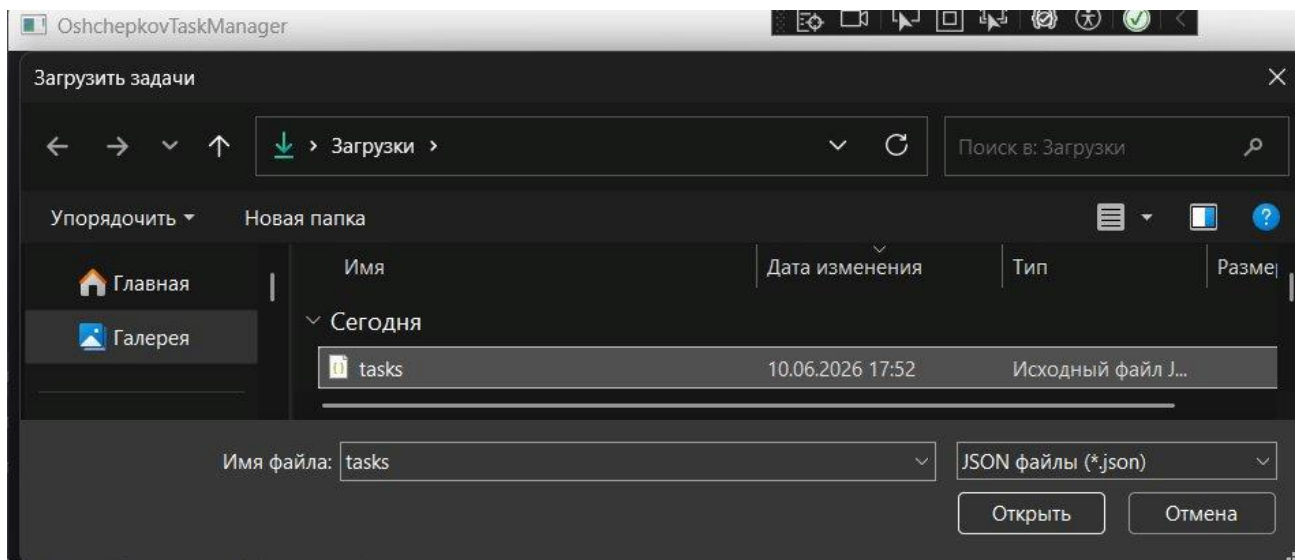


Рисунок 12 — Диалог загрузки задач из ранее сохранённого JSON-файла

6.8. Индикация просроченных задач

Задачи, срок выполнения которых истёк, при этом статус которых не является «Завершена» или «Отменена», автоматически выделяются в списке тёмно-красным фоном — это реализовано с помощью триггера DataTrigger, привязанного к вычисляемому свойству IsOverdue модели TaskItem. Количество просроченных задач отображается отдельным счётчиком в блоке статистики боковой панели.

Просроченная задача	Эта задача не была выполнена...	Низкий	Новая	07.06.2026	10.06.2026
---------------------	---------------------------------	--------	-------	------------	------------

Рисунок 13 — Отображение просроченной задачи в общем списке (выделена красным)

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2026 г. были закреплены теоретические знания и расширены профессиональные умения в области разработки программного обеспечения на платформе .NET с использованием языка программирования C#.

По итогам практики получены следующие результаты:

1. Разработано техническое задание на приложение для управления задачами OshchepkovTaskManager с полным перечнем функциональных требований;
2. Спроектирована и реализована архитектура приложения с разделением на библиотеку бизнес-логики (динамическую библиотеку OshchepkovTaskManager.Core) и пользовательский интерфейс (WPF-приложение OshchepkovTaskManager.App), построенный по паттерну MVVM (Model-View-ViewModel);
3. Реализованы все обязательные функции приложения: добавление, просмотр, редактирование и удаление задач, фильтрация по статусу, поиск по названию и описанию, сохранение и загрузка данных в формате JSON;
4. Дополнительно реализованы сортировка задач по приоритету и сроку выполнения, визуальная пометка важных задач, а также автоматический подсчёт статистики по задачам;
5. Построена диаграмма классов разработанной библиотеки средствами Visual Studio 2022, отражающая структуру и взаимосвязи классов и интерфейсов;
6. Выявлены и устранены ошибки конфликта имён типов, несовместимости свойств XAML-разметки с используемой версией WPF, проблемы отображения текста в элементах ввода, а также уязвимость используемого NuGet-пакета;

7. Разработаны и успешно выполнены 39 модульных тестов с использованием фреймворка xUnit, подтверждающих корректность реализации ключевых классов библиотеки;
8. Проведено инспектирование программного кода на соответствие стандартам кодирования C# и принципам объектно-ориентированного проектирования (в частности, принципам SOLID).

По окончании практики был получен практический опыт разработки требований к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

Таким образом, цель учебной практики — комплексное освоение видов профессиональной деятельности, предусмотренных профессиональным модулем ПМ.02 «Осуществление интеграции программных модулей» — достигнута в полном объёме. Полученные навыки работы с объектно-ориентированным программированием, коллекциями, файловым вводом-выводом, технологией LINQ, паттернами проектирования MVVM и Dependency Injection, а также с инструментами модульного тестирования xUnit, могут быть использованы при дальнейшем обучении и в будущей профессиональной деятельности.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — М.: Изд-во стандартов, 1981.
2. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. — М.: Стандартинформ, 2020.
3. ГОСТ 19.402-78. Единая система программной документации. Описание программы. — М.: Изд-во стандартов, 1978.
4. Троелсен Э., Джепикс Ф. Язык программирования C# 9 и платформа .NET 5. — М.: Вильямс, 2022. — 1328 с.
5. Мак-Дональд М. WPF: Windows Presentation Foundation в .NET 4.5 с примерами на C# 5.0 для профессионалов. — М.: Вильямс, 2013. — 1024 с.
6. Фримен Э. Паттерны проектирования. — СПб.: Питер, 2022. — 672 с.
7. Албахари Дж., Албахари Б. C# 9.0. Справочник. Полное описание языка. — М.: Вильямс, 2021. — 1008 с.
8. Мартин Р. Чистый код: создание, анализ и рефакторинг. — СПб.: Питер, 2019. — 464 с.
9. Документация Microsoft по платформе .NET и технологии WPF [Электронный ресурс]. — URL: <https://learn.microsoft.com/ru-ru/dotnet/> (дата обращения: 20.06.2026).
10. Документация фреймворка модульного тестирования xUnit.net [Электронный ресурс]. — URL: <https://xunit.net/docs/getting-started/netcore/cmdline> (дата обращения: 20.06.2026).
11. Руководство по стилю кода C# (C# Coding Conventions) [Электронный ресурс]. — URL: <https://learn.microsoft.com/ru-ru/>

ru/dotnet/csharp/fundamentals/coding-style/coding-conventions (дата обращения: 20.06.2026).

12. Документация по библиотеке System.Text.Json [Электронный ресурс].
— URL: <https://learn.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json/overview> (дата обращения: 20.06.2026).

ТРЕБОВАНИЕ К НАПИСАНИЮ ПРОГРАММНОГО МОДУЛЯ

Задание.

Практика работы с классами, коллекциями, файловым вводом-выводом, LINQ и основами ООП в C#.

Цель:

Разработать приложение для управления задачами, демонстрирующее владение объектно-ориентированным программированием на языке C#, технологией LINQ для обработки коллекций данных, организацией файлового ввода-вывода и базовыми принципами архитектуры программных систем.

1. Основные требования. Разработать приложение для управления задачами с возможностью:

13. Добавления новой задачи (название, описание, приоритет, срок выполнения, статус).
 14. Просмотра списка всех задач.
 15. Фильтрации задач по статусу (например, «Новая», «В процессе», «Завершена»).
 16. Поиска задач по названию или описанию.
 17. Редактирования существующих задач.
 18. Удаления задач.
 19. Сохранения и загрузки задач в/из файла (JSON или XML).
 20. Основная логика приложения должна находиться в динамической библиотеке, а пользовательский интерфейс должен представлять из себя WPF-приложение.
 21. Для классов в библиотеке необходимо написать модульные тесты.
2. Дополнительные задания (по желанию):

- Реализовать сортировку задач по приоритету или сроку выполнения.
- Добавить возможность пометки задачи как «Важная» (выводить звёздочкой или цветом).
- Реализовать простую статистику (например, сколько задач завершено, сколько просрочено).

Все перечисленные требования, как основные, так и дополнительные, были выполнены в полном объёме в ходе настоящей практики, что подтверждается описанием функциональности приложения в разделах 2 и 6 настоящего отчёта, а также результатами модульного тестирования, представленными в разделе 4.